Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: Mehdi Heidari, Narges Hosseini, Abolfazl Tadrisi, Arvin Afshari, Amir Arshia Tootoonchi, Fatemeh Feyzi

سوال ۱ (۱۰ امتیاز): تصحیح برگه‌های کوییز

کوییز اول درس داده ساختارها برگزار شده و برگه‌ها تصحیح شده‌اند. حالا نوبت شماست که به‌عنوان دستیار آموزشی، لیست نمره‌ها را مرتب کنید تا بتوان به‌سرعت وضعیت کلاس را بررسی کرد. استاد علاوه بر لیست مرتب‌شده، می‌خواهد بداند پرتکرارترین نمره‌ی کلاس چه بوده است.

ورودی

- سطر اول: یک عدد صحیح n، تعداد دانشجویان.
- سطر دوم: nعدد صحیح جدا شده با فاصله، نمره‌ی دانشجویان.

محدودیت‌ها:

- 1 <= n <= 300
- 0 <= score <= 100

خروجی

- سطر اول: nعدد صحیح جدا شده با فاصله؛ نمره‌ها به‌صورت صعودی مرتب شده.
- سطر دوم: پرتکرارترین نمره. اگر چند نمره بیشترین تکرار را داشتند، کوچکترین آن‌ها را چاپ کنید.

مثال

ورودی نمونه:

```
8
17 20 13 20 17 8 20 13
```

خروجی نمونه:

```
8 13 13 17 17 20 20 20
20
```

توضیح مثال: نمره‌ی ۲۰ سه بار، نمره‌های ۱۳ و ۱۷ هر کدام دو بار و نمره‌ی ۸ یک بار تکرار شده‌اند؛ پس پرتکرارترین نمره ۲۰ است.

سوال ۲ (۳۰ امتیاز): چیدن دست کارت

وقتی کارت‌های بازی را در دستتان می‌چینید، معمولاً کارت‌ها را یکی‌یکی برمی‌دارید و هر کارت جدید را در جای درست خودش بین کارت‌های مرتب‌شده جا می‌دهید؛ برای این کار مجبورید بخشی از کارت‌ها را کنار بزنید تا جا باز شود.

استاد از شما خواسته ابزاری بنویسید که این فرایند را برای دانشجویان قابل مشاهده کند و هزینه‌ی واقعی آن را هم اندازه بگیرد.

فرایندی که باید شبیه‌سازی کنید

آرایه را از چپ به راست پردازش می‌کنیم. در ابتدا فقط عنصر اندیس ۰ را «مرتب‌شده» در نظر می‌گیریم. سپس به ازای هر k از ۱ تا n-1 مرحله‌ی زیر انجام می‌شود:

عنصر اندیس k را برمی‌داریم و آن را در جای درست خودش داخل بخش مرتب‌شده‌ی سمت چپ قرار می‌دهیم. برای باز کردن جا، هر عنصری از بخش مرتب‌شده که اکیداً از آن بزرگتر است یک خانه به سمت راست منتقل می‌شود. به‌محض رسیدن به عنصری که از آن بزرگتر نیست (یا رسیدن به ابتدای آرایه) کار این مرحله تمام می‌شود.
بعد از پایان هر مرحله، وضعیت کامل آرایه را چاپ می‌کنید.
تعریف دقیق «انتقال»: هر بار که یک عنصر برای باز کردن جا، یک خانه به سمت راست منتقل می‌شود، یک واحد شمرده می‌شود.

ورودی

- سطر اول: یک عدد صحیح n، تعداد عناصر آرایه.
- سطر دوم: nعدد صحیح جدا شده با فاصله.

محدودیت‌ها:

- $2 <= n <= 20$
- $-999 <= A[i] <= 999$

خروجی

- سطر اول: عناصر جدا شده با فاصله (به ازای $k = 1$ تا $(n-1)$)، وضعیت آرایه پس از هر مرحله n-1.
- سطر آخر: یک عدد صحیح، تعداد کل انتقال‌ها.

مثال

ورودی نمونه:

```
5
5 2 4 1 3
```

خروجی نمونه:

```
2 5 4 1 3
2 4 5 1 3
1 2 4 5 3
1 2 3 4 5
7
```

توضیح مثال:

عدد ۲ باید از روی ۵ رد شود؛ ۱ انتقال. = k
1:
عدد ۴ باید از روی ۵ رد شود؛ ۱ انتقال. = k
2:
عدد ۱ باید از روی ۵ و ۴ و ۲ رد شود؛ ۳ انتقال. = k
3:
عدد ۳ باید از روی ۵ و ۴ رد شود ولی جلوی ۲ متوقف می‌شود؛ ۲ انتقال. :4 = k

در مجموع ۱ + ۱ + ۳ + ۲ = ۷ انتقال.

سوال ۳ (۵۰ امتیاز): حباب‌ها تا کی بالا می‌آیند؟

یک آرایه به شما داده شده است. روی این آرایه فرایند زیر را انجام می‌دهیم:
آرایه را در دورهای متوالی پیمایش می‌کنیم. در هر دور، از انتهای آرایه به سمت ابتدا حرکت می‌کنیم؛ یعنی به ازای j از n-1تا ۱، اگر عنصر اندیس j اکیداً کوچکتر از عنصر اندیس j-1 بود، جای آن دو را با هم عوض می‌کنیم.

دورها را تا وقتی تکرار می‌کنیم که یک دور کامل انجام شود و در تمام آن هیچ جابه‌جایی رخ ندهد. آن دور آخر هم یک دور کامل حساب می‌شود و باید شمرده شود.
باید گزارش دهید در کل این فرایند چند جابه‌جایی انجام شده، چند دور اجرا شده، و آرایه‌ی نهایی چه شکلی است.

ورودی

- سطر اول: یک عدد صحیح n، تعداد عناصر آرایه.
- سطر دوم: n عدد صحیح جدا شده با فاصله.

محدودیت‌ها:

- $1 <= n <= 300$
- $-9999 <= A[i] <= 9999$

خروجی

- سطر اول: تعداد کل جابه‌جایی‌ها.
- سطر دوم: تعداد دورهای اجراشده.
- سطر سوم: آرایه‌ی نهایی، عناصر جدا شده با فاصله.

مثال

ورودی نمونه:

```
6
3 1 2 6 5 4
```

خروجی نمونه:

```
5
3
1 2 3 4 5 6
```

توضیح مثال:

- دور اول: سه جابه‌جایی انجام می‌شود و آرایه به [1, 3, 2, 4, 6, 5] می‌رسد.
- دور دوم: دو جابه‌جایی انجام می‌شود و آرایه به [1, 2, 3, 4, 5, 6] می‌رسد.
- دور سوم: هیچ جابه‌جایی رخ نمی‌دهد و فرایند متوقف می‌شود.

پس تعداد دورهای اجراشده ۳ و تعداد کل جابه‌جایی‌ها ۵ است.

سوال ۴ (۷۰ امتیاز): مرتب‌سازی شماره‌های دانشجویی (مرتب‌سازی مبنایی)

Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: **Mehdi Heidari**, **Narges Hosseini**, **Abolfazl Tadrisi**, **Arvin Afshari**, **Amir Arshia Tootoonchi**,

آموزش دانشگاه لیستی از شماره‌های دانشجویی را به شما داده و می‌خواهد آن‌ها را مرتب کنید. شماره‌ها اعداد صحیح نامنفی هستند. چون این اعداد «طول محدود» دارند، بهترین انتخاب مرتب‌سازی مبنایی (Radix Sort) است که بدون حتی یک مقایسه‌ی بزرگتر یا کوچکتر ، فقط با نگاه کردن به رقم‌ها کار را تمام می‌کند.
استاد می‌خواهد مطمئن شود واقعاً مرتب‌سازی مبنایی را پیاده کرده‌اید و نه چیز دیگری.
بنابراین برنامه‌ی شما باید وضعیت آرایه را بعد از هر دور (یعنی بعد از مرتب‌سازی بر اساس یکان، بعد دهگان ، بعد صدگان و ...) چاپ کند.

الگوریتم دقیقی که باید پیاده‌سازی کنید

نسخه‌ی غیربازگشتی و از کم‌ارزش به پرارزش LSD :
- تعداد دورها را برابر تعداد ارقام بزرگترین عدد آرایه قرار دهید. (اگر بزرگترین عدد صفر باشد، یعنی همه‌ی عناصر صفرند، تعداد دورها ۱ است.)
- در هر دور، ۱۰ سطل خالی برای ارقام ۰ تا ۹ بسازید.
- مرحله‌ی پخش کردن : آرایه را از چپ به راست پیمایش کنید و هر عدد x را به انتهای سطل شماره‌ی (x // 10 % radix) اضافه کنید.
- مرحله‌ی جمع‌آوری: سطل‌ها را به ترتیب از ۰ تا ۹ پشت سر هم بچینید و آرایه‌ی جدید را بسازید.
- وضعیت آرایه را در پایان هر دور چاپ کنید، سپس radix را در ۱۰ ضرب کنید.

ورودی
- سطر اول: یک عدد صحیح n، تعداد شماره‌ها.
- سطر دوم: nعدد صحیح نامنفی جدا شده با فاصله.

محدودیت‌ها:

- $1 <= n <= 50$
- $0 <= A[i] <= 99999$

خروجی
به تعداد دورهای الگوریتم سطر چاپ کنید. سطرkام باید وضعیت آرایه بعد از دور kام باشد؛ عناصر جدا شده با فاصله. طبیعتاً سطر آخر همان آرایه‌ی کاملاً مرتب‌شده است.

مثال

ورودی نمونه:

```
6
170 45 75 90 802 2
```

خروجی نمونه:

```
170 90 802 2 45 75
802 2 45 170 75 90
2 45 75 90 170 802
```

توضیح مثال: بزرگترین عدد ۸۰۲ است که ۳ رقم دارد، پس ۳ دور اجرا می‌شود.

- دور اول (یکان) : ارقام یکان به ترتیب ۰، ۵، ۵، ۰، ۲، ۲ هستند. سطل ۰ شامل [170, 90] ، سطل ۲ شامل [802, 2] و سطل ۵ شامل [45, 75] است. جمع‌آوری این سطل‌ها آرایه‌ی سطر اول خروجی را می‌سازد.
- دور دوم (دهگان) : روی خروجی دور اول اجرا می‌شود؛ سطل ۰ شامل [802, 2] ، سطل ۴ شامل [45]، سطل ۷ شامل [170, 75] و سطل ۹ شامل [90] است.
- دور سوم (صدگان) : سطل ۰ شامل [2, 45, 75, 90] ، سطل ۱ شامل [170] و سطل ۸ شامل [802] است و آرایه کاملاً مرتب می‌شود.

سوال ۵ (۱۰۰ امتیاز): سامانه‌ی رتبه‌بندی کلاس (مرتبسازی پایدار و چندکلیدی)

در جزوه خواندیم که یک الگوریتم مرتبسازی پایدار (Stable) است اگر وقتی به دو عنصر با مقدار برابر می‌رسد، ترتیب اولیه‌ی آن‌ها را حفظ کند. همچنین دیدیم که مرتبسازی مبنایی دقیقاً روی همین خاصیت بنا شده است: با چند بار مرتبسازی پایدار پشت سر هم، می‌توان روی چند کلید مرتب کرد.
حالا می‌خواهیم همین ایده را در یک سامانه‌ی واقعی پیاده کنیم. سامانه‌ی رتبه‌بندی کلاس، لیستی از دانشجویان را نگه می‌دارد و به فرمان‌های شما پاسخ می‌دهد. هر دانشجو سه ویژگی دارد: نام، نمره و سن.

نکته‌ی کلیدی این است که همه‌ی دستورهای مرتب‌سازی باید پایدار باشند. به این ترتیب اگر ابتدا بر اساس سن و سپس بر اساس نمره مرتب کنید، نتیجه لیستی است که بر اساس نمره مرتب شده و دانشجویان هم‌نمره بین خودشان بر اساس سن مرتب‌اند؛ دقیقاً همان کاری که مرتب‌سازی مبنایی با رقم‌ها می‌کند.

دستورها

برنامه سطر به سطر دستور می‌خواند تا به دستور exit یا پایان ورودی برسد.

برنامه باید دستورات زیر را از ورودی بخواند و اجرا کند:

add <name> <score> <age>:

یک دانشجوی جدید را به انتهای لیست اضافه می‌کند و پیام Added <name>را چاپ می‌کند. اگر دانشجویی با همین نام از قبل در لیست وجود داشته باشد، پیام Name already exists چاپ می‌شود.

remove <name>:

دانشجوی موردنظر را از لیست حذف می‌کند و پیام Removed <name>را چاپ می‌کند. اگر چنین دانشجویی وجود نداشته باشد، پیام Record not found چاپ می‌شود.

sort <field>:

لیست را به‌صورت پایدار (Stable) و صعودی بر اساس فیلد داده‌شده مرتب می‌کند و پیام Sorted by <field> asc را چاپ می‌کند.

rsort <field>:

لیست را به‌صورت پایدار (Stable) و نزولی بر اساس فیلد داده‌شده مرتب می‌کند و پیام Sorted by <field> desc را چاپ می‌کند.

Show:

اگر لیست خالی باشد، پیام No records چاپ می‌شود. در غیر این صورت، ابتدا سطر :Records و سپس، به ترتیب فعلی لیست، اطلاعات هر دانشجو در یک سطر و به شکل <name> <score> <age> چاپ می‌شود.

exit:

برنامه پایان می‌یابد.

مقدار <field> باید یکی از سه مقدار name یا score یا age باشد.

تعریف دقیق پایداری در حالت نزولی

در دستور rsort ، عناصر بر اساس کلید به‌صورت نزولی مرتب می‌شوند، ولی عناصری که کلید یکسان دارند باید ترتیب فعلی خود در لیست را حفظ کنند. توجه کنید که این با «معکوس کردن نتیجه‌ی مرتب‌سازی صعودی» فرق دارد؛ معکوس کردن، ترتیب عناصر هم‌کلید را هم برعکس می‌کند و پاسخ شما را غلط خواهد کرد.

ورودی نامعتبر

در هر یک از حالت‌های زیر باید دقیقاً عبارت invalid input چاپ شود و برنامه به کار خود ادامه دهد:

- دستور ناشناخته باشد یا سطر خالی باشد.
- تعداد پارامترهای دستور درست نباشد.
- name شامل کاراکتری غیر از حروف انگلیسی، ارقام و زیرخط باشد، یا طول آن بیشتر از ۲۰ باشد.
- score عدد صحیح در بازه‌ی ۰ تا ۱۰۰ نباشد.
- age عدد صحیح در بازه‌ی ۱ تا ۱۲۰ نباشد.
- <field> یکی از سه مقدار مجاز نباشد.

توجه کنید که Name already exists و Record not found وضعیت‌های منطقی برنامه‌اند و ورودی نامعتبر محسوب نمی‌شوند.

مقایسه‌ی نام‌ها

نام‌ها بر اساس ترتیب الفبایی کدهای اسکی مقایسه می‌شوند؛ یعنی همان مقایسه‌ی معمول رشته‌ها. توجه کنید که در این ترتیب همه‌ی حروف بزرگ قبل از همه‌ی حروف کوچک می‌آیند.

محدودیت‌ها

- تعداد دستورها حداکثر ۶۰
- تعداد دانشجویان در هر لحظه حداکثر ۲۰

مثال

ورودی نمونه:

```
add Ali 18 21
add Sara 20 19
add Reza 18 25
add Mina 20 22
sort age
sort score
show
rsort score
show
exit
```

خروجی نمونه:

```
Added Ali
Added Sara
Added Reza
Added Mina
Sorted by age asc
Sorted by score asc
Records:
Ali 18 21
Reza 18 25
Sara 20 19
Mina 20 22
Sorted by score desc
Records:
Sara 20 19
Mina 20 22
Ali 18 21
Reza 18 25
```

توضیح مثال: بعد از sort age لیست به ترتیب Sara(19)، Ali(21)، Mina(22)، Reza(25) می‌شود. سپس sort score روی همین لیست اجرا می‌شود و چون پایدار است، بین دو نفر با نمره‌ی ۱۸ ترتیب سنی (Aliقبل از Reza) و بین دو نفر با نمره‌ی ۲۰ هم ترتیب سنی (Saraقبل از Mina)حفظ می‌شود. در پایان rsort score فقط ترتیب گروه‌های نمره را برعکس می‌کند، ولی ترتیب داخل هر گروه دست‌نخورده می‌ماند.